

BUILD-CONVENTIONS.md

Version : V1

Statut : Référence officielle des conventions de développement

Type : Build Blueprint

1. Objectif

Ce document définit les conventions obligatoires de développement pour l'ensemble du projet Vitala.

Son objectif est de garantir :

- une architecture cohérente ;
- un code homogène ;
- une maintenance facilitée ;
- une génération de code déterministe par les développeurs et les IA.

Ces conventions sont obligatoires pour tous les modules du projet.

2. Principes généraux

Chaque développement doit respecter les principes suivants :

- Domain First
- Clean Architecture
- API First
- Offline First
- Security by Design
- Documentation First
- Test First lorsque applicable

Toute dérogation doit être documentée et approuvée.

3. Conventions de nommage

Fichiers

- noms explicites ;
- minuscules ;
- mots séparés par des tirets ou des underscores selon le langage utilisé ;
- aucun nom abrégé non documenté.

Exemples :

- flash_service
 - mission_repository
 - trust_controller
-

Classes

Les classes utilisent le format PascalCase.

Exemples :

- FlashService
 - MissionRepository
 - RecommendationEngine
-

Variables

Les variables utilisent le format camelCase.

Leur nom doit refléter clairement leur rôle.

Les abréviations ambiguës sont interdites.

Constantes

Les constantes utilisent le format UPPER_SNAKE_CASE.

Exemple :

DEFAULT_SYNC_INTERVAL

4. Organisation des dossiers

Chaque domaine possède sa propre structure.

Exemple logique :

- domain
- application
- infrastructure
- presentation
- tests

Aucun fichier ne doit être placé hors de son domaine fonctionnel.

5. Organisation des modules

Chaque module doit contenir uniquement les éléments qui lui appartiennent.

Il est interdit :

- de partager directement des modèles métier entre domaines ;
 - de créer des dépendances circulaires ;
 - de placer de la logique métier dans les composants UI.
-

6. Gestion des dépendances

Les dépendances suivent exclusivement la hiérarchie officielle :

Présentation → Application → Domaine → Infrastructure → Plateforme.

Toute dépendance inverse est interdite.

7. Gestion des APIs

Chaque API doit :

- appartenir à un domaine ;
- respecter son contrat officiel ;
- être versionnée ;
- retourner des réponses cohérentes ;
- documenter ses erreurs.

Aucune API ne doit contourner un domaine métier.

8. Gestion de la base de données

Toutes les tables doivent :

- appartenir à un seul domaine ;
- posséder une clé primaire ;
- inclure les champs d'audit requis ;
- respecter les politiques RLS ;
- être créées via des migrations versionnées.

Les modifications directes en production sont interdites.

9. Gestion des migrations

Chaque évolution de la base de données doit être réalisée par une migration.

Une migration doit :

- être atomique ;
 - être réversible lorsque possible ;
 - être documentée ;
 - être testée avant déploiement.
-

10. Gestion du code

Chaque fichier doit respecter une responsabilité unique.

Les fichiers trop volumineux doivent être découpés.

La duplication de logique est interdite.

Les fonctions doivent rester courtes, lisibles et testables.

11. Gestion des tests

Chaque domaine doit comporter :

- des tests unitaires ;
- des tests d'intégration lorsque nécessaire.

Les fonctionnalités critiques (authentification, synchronisation, Trust, UDI) doivent disposer d'une couverture renforcée.

Une fonctionnalité n'est pas considérée comme terminée tant que les tests associés ne sont pas validés.

12. Gestion de Git

Les règles suivantes sont obligatoires :

- une branche par fonctionnalité ;
- des commits atomiques ;
- des messages de commit explicites ;

- aucune modification directe sur la branche principale.

Chaque Pull Request doit être relue avant fusion.

13. Documentation

Toute modification impactant :

- un domaine ;
- une API ;
- une table ;
- une règle métier ;
- une architecture ;

doit entraîner la mise à jour de la documentation concernée.

Le code et la documentation doivent rester synchronisés.

14. Gestion des erreurs

Toutes les erreurs doivent :

- être typées ;
- être journalisées ;
- être compréhensibles ;
- être converties en réponses utilisateur adaptées.

Les messages techniques internes ne doivent jamais être exposés aux utilisateurs.

15. Sécurité

Les règles suivantes sont obligatoires :

- validation systématique des entrées ;
 - contrôle des permissions ;
 - application des politiques RLS ;
 - utilisation exclusive des APIs officielles ;
 - protection des données sensibles.
-

16. Qualité du code

Le code doit être :

- lisible ;
- modulaire ;
- documenté lorsque nécessaire ;
- facilement testable ;
- réutilisable.

Les optimisations prématurées sont à éviter.

17. Revue de code

Avant validation, chaque développement doit être vérifié selon les critères suivants :

- respect de l'architecture ;
 - respect des conventions ;
 - absence de duplication ;
 - qualité des tests ;
 - conformité aux contrats API ;
 - conformité au Domain Model.
-

18. Règles spécifiques pour les IA

Toute IA générant du code pour Vitala doit :

- respecter l'architecture officielle ;
 - respecter les conventions de ce document ;
 - consulter les API Contracts avant toute implémentation ;
 - ne jamais inventer une structure différente de celle définie dans le Build Blueprint ;
 - produire un code cohérent avec les documents officiels.
-

19. Critères de conformité

Une implémentation est conforme uniquement si :

- toutes les conventions sont respectées ;
 - aucune règle d'architecture n'est violée ;
 - le code est documenté lorsque nécessaire ;
 - les tests sont validés ;
 - la documentation est maintenue à jour.
-

20. Conclusion

BUILD-CONVENTIONS.md constitue la référence officielle des pratiques de développement de Vitala.

Il s'applique à tous les développeurs, à toutes les équipes et à toutes les intelligences artificielles intervenant sur le projet.